Reference Implementation & Software Overview (SecretML v4)

4.1 The Functional Prototype: SecretML v4

The current reference implementation, SecretML v4, serves as a stable, cross-platform proof of concept (optimized for Windows 10+). It features a dedicated graphical user interface (GUI) designed to facilitate secure key generation without compromising Zero-Knowledge integrity.

Core Operational Principles:

- **Zero-Persistence State:** The application operates in a strictly volatile environment. No sensitive data is ever cached to local databases or external servers.
- **Memory Isolation:** All critical contexts — questions, answers, and secret payloads — exist only within RAM for the duration of the active session.
- **Active Sanitization:** Upon completion or manual reset, the `reset_user_data()` routine purges all cryptographic material from memory.

4.2 The Key Derivation Pipeline

SecretML v4 utilizes a multi-stage pipeline to forge a 256-bit Master Key. The system supports two primary derivation modes to adapt to different threat levels:

4.2.1 Linear Entropy Fusion (Standard)

The baseline protocol optimized for speed. It layers cognitive responses with a physical file hash into a single, memory-hard derivation cycle.

$K_{combined} = H(Ans_0 + Salt_{file}) + \sum_{i=1}^{n} H(Ans_i)$
$K_{final} = H(K_{combined})$ Where H represents the SHA-256 hashing function.

4.2.2 Phantom-Step Cascade (High-Assurance)

An advanced engine based on **Sequential Dependency**. In this mode, the system realizes a step-wise decryption cascade: data for Step N does not exist in a readable state until Step $N-1$ is successfully unlocked.

$k_0 = Hash(SecretFile)$
$k_i = Argon2id(\ Answer_i,\ Salt = k_{i-1}\ )$
$K_{final} = Argon2id(\ k_n,\ Salt = Internal_Pepper\ )$ Each step i acts as the encryption key for the next prompt's metadata.

Technical Advantages:

- **Parallelization Resistance:** Since step i requires the output of $i-1$, attackers cannot distribute the workload across GPU cores.
- **Chain Integrity:** Any single error in the sequence renders the final payload permanently inaccessible, providing a robust defense against forensic analysis.

4.3 AI-Enhanced Security: "Cognitive Assistance"

SecretMemoryLocker integrates Large Language Models (LLMs) to overcome the "Creative Block" of security questions.

- **AI-Curated Database:** v4 includes 1,000+ pre-generated questions categorized by difficulty and theme.
- **Dynamic Question Forge (Upcoming):** A local LLM assistant to help users craft highly personal, high-entropy questions that are easy to remember but impossible for others to guess.

4.4 Cryptographic Primitives & Stack

We rely exclusively on industry-standard, audited libraries for the reference implementation:

COMPONENT	PRIMITIVE / LIBRARY	PURPOSE
KDF	Argon2id	Memory-hard deterministic key generation.
Hashing	SHA-256	Integrity verification and entropy stacking.
Encryption	ChaCha20-Poly1305	Authenticated encryption (AEAD) for payloads.
Interface	Tkinter / ttk	Lightweight, native GUI for Windows environment.

4.5 Development Roadmap: Hardening & Expansion

SecretML v4 is a concept validation tool. The roadmap for production-grade security includes:

Software & Audit:

- **Memory Hardening:** Porting core logic to **Rust** to eliminate Python's memory management vulnerabilities.
- **MirageLoop:** Automating the generation of "decoy" sequences to misdirect automated tools.

Entropy Anchor Evolution:

- **Biometric Integration:** Replacing file salts with non-reproducible 256-bit vectors from FaceID, TouchID, or Iris scans.
- **Hardware Tokens:** Native support for YubiKey (U2F) and e-ID signatures as primary physical anchors.

Security Stance:

The architecture ensures that raw biometric data or hardware secrets are **never stored**. Only their one-way cryptographic representations enter the hashing pipeline.